

Instituto Tecnológico de San Luis Potosí

Ingeniería en Sistemas Computacionales

Lenguajes de Interfaz

11:00-12:00

Tarea 1 de 3 Unidad 1

Actividad 2 de 3 Unidad 1

Alumna:

Venegas Ordaz Fátima Vianney

Lugar y Fecha:

Soledad de Graciano Sánchez, San Luis Potosí, 03 de febrero de 2026

Índice

Índice de Imágenes	1
Introducción.	2
Marco Teórico.....	3
¿Qué es un lenguaje de alto nivel y que es un lenguaje de bajo nivel?	3
¿Qué es un microprocesador?	4
¿Qué es un microcontrolador?	4
¿Qué es una microcomputadora?	5
¿Qué es un sistema mínimo y que elementos lo componen?	6
Entornos de Desarrollo Integrado (IDE) para Lenguaje Ensamblador... ..	7
Conclusión.....	9
Referencias.....	10

Índice de Imágenes

Imagen 1. Microprocesador	4
Imagen 2. El microcontrolador y sus componentes.	5

Introducción.

La presente investigación busca entender y analizar a los pilares de la computación como lo viene siendo la diferencia entre un lenguaje de alto y bajo nivel, a su vez se busca entender lo que es un microcontrolador, microprocesador y un microcomputador y en que se diferencian en su mayoría.

Además de que también se analiza lo que es un sistema mínimo y como este es fundamental para que un procesador sea funcional y pueda ejecutar instrucción. Como objetivo principal está el comprender la arquitectura fundamental que permite la interacción entre lo que es el hardware con el procesamiento de datos. Así mismo identificar las características técnicas de cada IDE y lograr entender como facilitan el proceso de ensamblado, enlace y depuración, en distintas arquitecturas de procesadores.

Marco Teórico.

¿Qué es un lenguaje de alto nivel y que es un lenguaje de bajo nivel?

Lenguaje de alto nivel.

Los lenguajes de programación de alto nivel se caracterizan porque su estructura semántica es muy similar a la forma como escriben los humanos, lo que permite codificar los algoritmos de manera más natural, en lugar de codificarlos en el lenguaje binario de las máquinas, o a nivel de lenguaje ensamblador. (Marin Monterde, s.f.)

Ventaja: Fácil de aprender, escribir y mantener; es portable entre diferentes sistemas.

Desventaja: Requiere ser "traducido" (compilado o interpretado), lo que puede consumir más recursos que el bajo nivel.

Lenguaje de bajo nivel.

Lenguaje de bajo nivel se refiere a un tipo de lenguaje de programación que está más cerca del código máquina y el hardware que los lenguajes de alto nivel. Proporciona control directo sobre el hardware y los recursos de la computadora, permitiendo a los programadores escribir código a un nivel más granular. Este tipo de lenguaje se utiliza normalmente para tareas que requieren un control preciso y una ejecución eficiente. (Lenovo Mexico, s.f.)

¿Por qué usar un lenguaje de bajo nivel?

Usaría un lenguaje de bajo nivel cuando necesite tener un control preciso sobre los recursos de hardware o cuando desee optimizar el rendimiento de su código. Los lenguajes de bajo nivel permiten manipular directamente la memoria, los registros y otros componentes de hardware, lo que brinda una capacidad de crear programas altamente eficientes y especializados. (Lenovo Mexico, s.f.)

Ventaja: Control total del hardware y alta eficiencia.

Desventaja: Difícil de leer y no es portable (un código para un procesador Intel no funciona en un ARM).

¿Qué es un microprocesador?

Según (Ramírez, 1986) Un **microprocesador** es un **circuito integrado** (chip) de alta o muy alta escala de integración (LSI o VLSI) que contiene todos los elementos necesarios para conformar una **Unidad Central de Procesamiento (CPU)**.

Es un dispositivo que realiza las funciones de la **CPU** en un único circuito integrado. Se pone en marcha cuando inicias tu ordenador y se encarga de activar el sistema operativo y los programas correspondientes. (Ramírez, 1986)

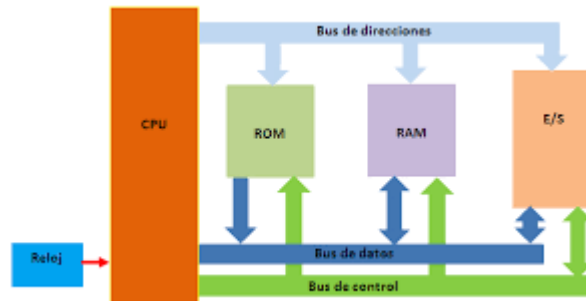


Imagen 1. Microprocesador

Características Principales:

Integración: Reúne en un solo componente la Unidad Aritmético-Lógica (ALU), la Unidad de Control y los Registros internos.

Función: Actúa como el "cerebro" del sistema, encargado de ejecutar las instrucciones de los programas y procesar los datos. (Ramírez, 1986)

¿Qué es un microcontrolador?

Según (Valdés Pérez & Pallàs Areny, 2007) Un **microcontrolador** es un **computador completo** en un solo circuito integrado de dimensiones reducidas. A diferencia de un microprocesador común, este chip contiene todos los elementos esenciales para funcionar de forma autónoma:

- **Unidad Central de Procesamiento (CPU):** Encargada de ejecutar las instrucciones.
- **Memoria:** Incluye tanto memoria de programa (ROM, EPROM o Flash) como memoria de datos (RAM) dentro del mismo chip.
- **Líneas de Entrada/Salida (I/O):** Puertos para comunicarse directamente con el mundo exterior.
- **Recursos Auxiliares:** Suele integrar temporizadores (timers), convertidores analógico-digitales y otros periféricos especializados.

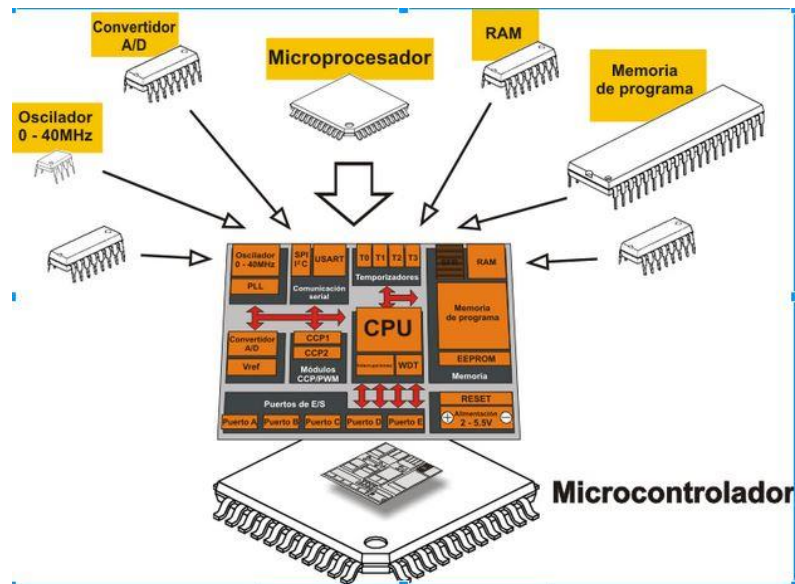


Imagen 2. El microcontrolador y sus componentes.

¿Qué es una microcomputadora?

Una microcomputadora es un dispositivo informático compacto y autónomo. Normalmente incluye un microprocesador, memoria y periféricos de entrada/salida. Estas computadoras, también conocidas como computadoras personales, están diseñadas para uso individual, lo que las hace versátiles y ampliamente utilizadas en diversas aplicaciones. (Lenovo, s.f.)

¿Qué es un sistema mínimo y que elementos lo componen?

Un **sistema mínimo** es la configuración básica y más pequeña de componentes electrónicos interconectados que permiten que un microprocesador sea funcional y pueda ejecutar instrucciones de un programa. Sin estos elementos, el procesador no tiene forma de recibir energía sincronizada, leer código o interactuar con datos, por lo que se considera el "esqueleto" operativo de cualquier computadora o sistema embebido.

Elementos que lo Componen

Para que un microprocesador funcione, requiere al menos de estos cinco bloques fundamentales:

- **Microprocesador (CPU):** Es el componente central que procesa los datos y ejecuta las instrucciones del software.
- **Generador de Reloj (Oscilador):** Proporciona una señal de impulsos eléctricos constantes que sincroniza todas las operaciones internas del procesador.
- **Memoria Permanente (ROM/EPROM/Flash):** Contiene el programa o las instrucciones iniciales que el procesador debe leer al encenderse.
- **Memoria de Acceso Aleatorio (RAM):** Se utiliza para almacenar datos temporales y resultados intermedios durante la ejecución de los procesos.
- **Circuito de Reset:** Permite inicializar el contador del programa y los registros del procesador a un estado conocido al momento de recibir energía.
- **Buses de Comunicación:** Líneas físicas (cables o pistas de circuito) que transportan las direcciones, los datos y las señales de control entre todos los componentes.

Ejemplo:

El Intel 8086 en "Modo Mínimo": Este procesador tiene un pin llamado MN/MX. Si lo conectas a voltaje alto (Vcc), el chip se configura para trabajar en un

sistema sencillo donde él mismo genera todas las señales de control para la memoria, sin necesidad de chips controladores extra complejos. (Ramírez, 1986)

Entornos de Desarrollo Integrado (IDE) para Lenguaje Ensamblador.

WinAsm.

WinAsm Studio es un entorno de desarrollo integrado (IDE) gratuito para desarrollar programas en Windows 32-bit y DOS 16-bit utilizando Microsoft Macro Assembler MASM y FASM utilizando el Add-In para FASM.

Easy Code

Entorno visual de desarrollo en lenguaje ensamblador.

Easy Code es el entorno visual de programación en ensamblador hecho para generar aplicaciones de 32 bits para Windows. La interfaz de Easy Code, muy parecida a la de Visual Basic, le permite programar una aplicación en ensamblador de manera rápida y fácil como nunca antes había sido posible. (Calzada Terrazas, 2017)

Visual Studio (con MASM)

Es la opción profesional de Microsoft para desarrollo en sistemas Windows.

- **Arquitecturas:** x86, x64, ARM.
- **Características:** Utiliza el Microsoft Macro Assembler (MASM). Es extremadamente potente y permite mezclar código de C++ con ensamblador, algo común en la programación de sistemas y controladores. (Microsoft, s.f.)

SASM (Simple Assembler IDE)

Es uno de los entornos más modernos y recomendados para estudiantes.

- **Arquitecturas:** x86 y x64.
- **Ensambladores soportados:** NASM, MASM, GAS y FASM.

- **Características:** Interfaz limpia basada en pestañas y un depurador visual que muestra los registros de forma sencilla. Es multiplataforma (Windows y Linux). (Manushin, 2025)

MARS (MIPS Assembler and Runtime Simulator)

- **Arquitecturas:** MIPS32.
- **Características:** Escrito en Java, es muy ligero y proporciona una visualización clara del camino de datos (datapath) y la segmentación de instrucciones. (Sanderson & Vollmar, 2025)

Emu8086

Aunque está limitado a la arquitectura de 16 bits del Intel 8086, sigue siendo el estándar académico.

- **Arquitecturas:** Intel 8086.
- **Características:** Incluye un emulador que permite ver cómo se ejecutan las instrucciones paso a paso y cómo afectan a los registros y a la memoria RAM. (Softonic, 2010)

RadASM:

Un IDE muy flexible que soporta una gran variedad de ensambladores (MASM, TASM, FASM, NASM, entre otros). (Calzada Terrazas, 2017)

Conclusión.

Al finalizar la investigación, se llegó a la conclusión de que es importante conocer las diferencias fundamentales entre lo que es un lenguaje de alto y bajo nivel, así como entre los microcontroladores, microprocesadores y microcomputadores ya que este conocimiento técnico nos permite seleccionar la herramienta que mejor se adapte a la complejidad y el propósito de cada diseño asegurando que el hardware cuente con el sistema mínimo para procesar datos de manera eficiente.

Mientras que la elección de un buen IDE para lenguaje ensamblador depende totalmente de la arquitectura del procesador y del objetivo del desarrollo. Tener un buen dominio de estos entornos es fundamental para cualquier ingeniero en sistemas ya que nos permiten optimizar el uso de los recursos más básicos de las computadora.

Referencias

- Calzada Terrazas, I. (22 de Agosto de 2017). *IDE'S PARA PROGRAMAR EN LENGUAJE ENSAMBLADOR*. Obtenido de <https://irenect.wordpress.com/2017/08/22/ides-para-programar-en-lenguaje-ensamblador/>
- Lenovo. (s.f.). *Lenovo Glossary*. Obtenido de <https://www.lenovo.com/in/en/glossary/microcomputer/>
- Lenovo Mexico. (s.f.). *Lenovo*. Obtenido de Lenguaje de Bajo Nivel: <https://www.lenovo.com/mx/es/glosario/lenguaje-bajo-nivel/>
- Manushin, D. (2025). *SASM - Simple crossplatform IDE*. Obtenido de <https://dman95.github.io/SASM/english.html>
- Marin Monterde, U. (s.f.). *Lenguajes de programación [Unidad de Apoyo para el Aprendizaje]*. Obtenido de Facultad de Contaduría y Administración, UNAM.: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/2655/mod_resource/content/1/UAPA-Lenguajes-Programacion/index.html
- Microsoft. (s.f.). *Visual Studio IDE y Microsoft Macro Assembler (MASM)*. Obtenido de <https://visualstudio.microsoft.com/>
- Ramírez, E. V. (1986). Introducción a los microprocesadores: equipo y sistemas. En E. V. Ramírez, *Introducción a los microprocesadores: equipo y sistemas* (pág. 12). RWM Online.
- Sanderson, P., & Vollmar, K. (2025). *MARS (MIPS Assembler and Runtime Simulator)*. Obtenido de <https://dpetersanderson.github.io/>
- Softonic. (2010). *Emu8086 - 8086 Microprocessor Emulator (Versión 4.08)*. Obtenido de <https://emu8086-microprocessor-emulator.softonic.com/>

Valdés Pérez, F. E., & Pallàs Areny, R. (2007). *Microcontroladores: Fundamentos y aplicaciones con PIC*. Barcelona: Marcombo.